Assignment-2

PART-2

Exercise 1

The MAC OPERATION ARCHITECTURE

$$Y = W.X + B$$

→ W, Y = Input Matrix

→ B = bias Matrix

1.1) Sanity checking

⇒ W = (n x n)

⇒ X = (n x p)

⇒ B = (M x p or x 1)

So, valid only if:

⇒ Columns of W Equal (=) to Rows of X

⇒ rows of W equal (=) to Rows of B

⇒ Colums of X Equal (=) to Colums of B

⇒ Pseudocode

```
if (W_cols != X_rows):
    RAM[0] = -1                    // Error
    STOP
if (W_rows != B_rows):
    RAM[0] = -1
    stop

if (X_cols != B_cols):
    RAM[0] = -1
    stop
```

RAM[0] = 1 // valid
continue Execution

So, W colums = X rows are needed for Matrix Multiplication

Y Size = (m x p) here the Bias must Match output size

so. B must be same size as Y

RAM[0] = -1 → Error

RAM[0] = 1 → success.

The sanity check verifies that the number of colums of W equal the numbers of row of X.

1.2 Approach A → The MATMUL Function call

1.3 Approach B → The ROWXCOL Funetion call

Exercise 2

1) Stack Usage:

* Approach A (MATMUL)

↳ uses one Function call for the entire computation

↳ stack is used mainly for:

↳ passing initial argument (pointer + dimension)

↳ returning the Result pointer.

↳ Low stack growth → Fewer push / pop, fewer call / return frames.

↳ Lower overhead, better performances on Hack

Approach B [RowxCol]

↳ calls RowxCol() for every output element Y[i][j]

↳ Each call.

↳ pushes argument (row_ptr, col_ptr, length)

↳ creates a call frame

↳ returns a value

↳ High stack usage → many push/pop + Frequent call/return

So, A → Low stack usage, Low over head

↳ Efficient

B → High stack usage, high overhead

↳ Inefficient.

2. Segment Usage:

Approach A → uses segments efficiently and locally

↳ argument, local → loop variable (i, j, k, sum)

↳ Input pointers (W, x, B, dims)

→ minimal temp usage in Approach A

→ Better data locality.

Approach B → Frequent function calls cause repeated use of argument, local → (new frame every call) and temp

→ Requires recomputing addresses repeatedly

→ In Approach B there is more segment traffic and memory access.

so,

Approach A → Better Segment locality, fewer accesses.

and

Approach B → more Segment usage, more memory movements.

3. Optimization Approach A

In approach A everything is inside one Function, so we can optimize loops and memory access.

Software level

→ Reduce loop control for inner loop (k).

→ Replace repeated `i*N+k` add-ress calculations with incre-mental pointers.

→ Keep frequently used values in registers instead of RAM

→ Add B[i][j] during accumul-lation to avoid extra pass.

Hardware -

→ iterate W row-wise and X column-wise consistently.

→ reuse loaded values when possible

* Faster Execution due to fewer memory OPS + fewer loop checks.

4) Optimization for Approach B

In Approach B [ROW×COL] the problem is too many Function calls, optimzation aim to reduce Call Overhead.

Here the compiler-level optimization

→ Replace `ROWXCOL()` calls with its body → Removes call/return overhead

→ Merge loops to reduce repeated setup work.

→ Avoid recomputing Address like row-ptr+k, col-ptr + k*p

→ Reduce push/pop operations:

↳ pass parameters via Fixed register/location instead of stack when possible

→ pointer increment instead of recompute

↳ move pointers directly instead of recalculating indices.

Report

Extra credit

♫ → Jaiye Sajana (Dhurandhar The Revenge.)